
Matemáticas Discretas 2 II

APLICACIÓN DEL ALGORITMO DE EUCLIDES PARA
PROBLEMAS COTIDIANOS

El problema de las dos jarras

Ecuaciones diofánticas, máximo común divisor y
búsqueda en el espacio de estados

Julián Camilo Londoño Zarate
Jarol David Zuluaga Grau
Jose Alfredo Quevedo Suarez
Samuel Mejía Ortiz

Documento técnico del proyecto

1

Introducción y motivación

La teoría de números es una de las ramas más antiguas de las matemáticas y, al mismo tiempo, una de las que más aplicaciones tiene en la computación moderna. Herramientas aparentemente sencillas, como el **máximo común divisor (MCD)** y el **algoritmo de Euclides**, constituyen la base de áreas tan diversas como la criptografía de clave pública, la aritmética modular, la generación de números pseudoaleatorios y la resolución de ecuaciones diofánticas. Comprender estos fundamentos permite conectar un contenido teórico con problemas concretos del mundo real.

La motivación de este proyecto es mostrar que un resultado clásico y elegante, la **identidad de Bézout**, obtenida a partir del algoritmo de Euclides, resuelve por completo un rompecabezas cotidiano: el **problema de las dos jarras**. Este acertijo, popularizado en la cultura general, no es solo un juego: representa una familia de situaciones en las que debemos medir, repartir o combinar cantidades enteras usando recursos de capacidad limitada. El mismo razonamiento aparece al dosificar ingredientes, repartir recursos en partes exactas, diseñar engranajes o analizar estados en teoría de juegos.

El proyecto integra tres perspectivas complementarias vistas en el curso: un **modelo matemático** (la ecuación diofántica lineal asociada), un **modelo algorítmico** (Euclides clásico y extendido, más una búsqueda en grafos) y una **implementación computacional** que decide la existencia de solución y, cuando existe, construye la secuencia concreta de operaciones.

2

Problema abordado

Se dispone de dos jarras sin marcas de medición, de capacidades enteras n y m litros. Partiendo de ambas vacías, y usando únicamente las operaciones permitidas, se desea determinar si es posible **medir exactamente k litros** y, en caso afirmativo, obtener la secuencia de pasos que lo logra.

Operaciones permitidas

- **Llenar** una jarra por completo a partir de una fuente ilimitada.
- **Vaciar** una jarra por completo.
- **Trasvasar** el contenido de una jarra a la otra hasta que la fuente quede vacía o la jarra receptora se llene (lo que ocurra primero).

Formalmente, el estado del sistema es un par ordenado (x, y) , donde x es el volumen actual de la jarra A e y es el volumen de la jarra B, sujeto a $0 \leq x \leq n$ y $0 \leq y \leq m$. El problema, en su versión de decisión, puede relacionarse con la ecuación:

$$n \cdot l + m \cdot j = k, \quad \text{con } l, j \in \mathbb{Z}$$

donde los coeficientes enteros l y j representan, intuitivamente, cuántas veces se llena o se vacía cada jarra (los valores positivos y negativos modelan llenados y vaciados netos). La pregunta central es, por tanto: **¿para qué valores de k tiene solución esta ecuación**, y cómo se traduce esa solución en acciones físicas sobre las jarras?

3.1 Divisibilidad y máximo común divisor

Dados dos enteros a y b , se dice que **d divide a a** (escrito $d \mid a$) si existe un entero c tal que $a = d \cdot c$. El **máximo común divisor** $\gcd(a, b)$ es el mayor entero positivo que divide simultáneamente a a y a b . Este valor es la pieza clave para decidir la solubilidad del problema.

3.2 Algoritmo de Euclides

El algoritmo de Euclides calcula el MCD de forma eficiente apoyándose en la siguiente propiedad fundamental de la división entera:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

El procedimiento reemplaza repetidamente el par (a, b) por $(b, a \bmod b)$ hasta que el segundo componente sea 0; en ese punto el primer componente es el MCD. El algoritmo siempre termina porque los residuos forman una sucesión estrictamente decreciente de enteros no negativos. Su costo es $O(\log(\min(a, b)))$ divisiones, un resultado relacionado con el teorema de Lamé y la sucesión de Fibonacci.

3.3 Algoritmo de Euclides extendido e identidad de Bézout

La **identidad de Bézout** afirma que, para cualesquiera enteros a y b , existen enteros x, y (los coeficientes de Bézout) tales que:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

El **algoritmo de Euclides extendido** no solo calcula $\gcd(a, b)$, sino que además obtiene explícitamente esos coeficientes x e y , arrastrando las combinaciones lineales a lo largo de las divisiones sucesivas. Más aún, el conjunto de todos los valores que pueden expresarse como $a \cdot x + b \cdot y$ con x, y enteros es exactamente el conjunto de **múltiplos de $\gcd(a, b)$** . Esta observación es la que resuelve por completo el problema.

3.4 Ecuaciones diofánticas lineales

Una ecuación diofántica lineal en dos incógnitas tiene la forma $a \cdot x + b \cdot y = c$ y se buscan soluciones **enteras**. El teorema central establece el criterio de existencia:

$$a \cdot x + b \cdot y = c \text{ tiene solución entera} \Leftrightarrow \gcd(a, b) \mid c$$

Si (x_0, y_0) es una solución particular y $d = \gcd(a, b)$, entonces todas las soluciones se generan mediante la familia paramétrica $x = x_0 + (b/d) \cdot t$, $y = y_0 - (a/d) \cdot t$, con $t \in \mathbb{Z}$. Esto nos permite ajustar una solución teórica para que respete las restricciones físicas del problema (capacidades y volúmenes no negativos).

3.5 Modelado del problema de las jarras

Aplicando lo anterior al problema, se obtiene el **criterio de solubilidad**: es posible medir exactamente k litros si y solo si se cumplen dos condiciones.

$$(1) \gcd(n, m) \mid k \quad (2) 0 \leq k \leq \max(n, m)$$

La condición (1) proviene de la identidad de Bézout: cualquier volumen alcanzable en una jarra es una combinación entera $n \cdot x + m \cdot y$. Por lo tanto, un múltiplo de $\gcd(n, m)$. La condición (2) es una restricción física evidente: no se puede almacenar en una sola jarra más de lo que cabe en la de mayor capacidad. Cuando $\gcd(n, m) = 1$, las jarras son **coprimas** y puede medirse cualquier cantidad entera entre 0 y $\max(n, m)$, como en el caso clásico de las jarras de 5 y 3 litros.

4 Diseño de la solución y desarrollo

La solución se diseñó en dos capas que se complementan: una capa **algebraica**, que decide rápidamente si el problema tiene solución, y una capa **constructiva**, que genera la secuencia real de operaciones.

4.1 Capa algebraica (decisión)

Se calcula $d = \gcd(n, m)$ con el algoritmo de Euclides y se evalúa el criterio de solubilidad. El algoritmo de Euclides extendido aporta, además, los coeficientes de Bézout, que sirven como justificación matemática de por qué la cantidad es alcanzable y ofrecen una estrategia teórica de llenados y vaciados. Esta capa responde en tiempo logarítmico, sin necesidad de explorar estados.

4.2 Capa constructiva (búsqueda en el espacio de estados)

Para obtener la secuencia mínima de pasos, se modela el problema como un **grafo dirigido implícito**: cada nodo es un estado (x, y) y cada arista es una de las seis operaciones posibles (llenar A, llenar B, vaciar A, vaciar B, verter $A \rightarrow B$, verter $B \rightarrow A$). Al recorrer este grafo con **búsqueda en anchura (BFS)** desde el estado inicial $(0, 0)$, la primera vez que se alcanza un estado con k litros en alguna jarra se obtiene, por construcción, el camino de **longitud mínima** (menor número de operaciones).

La combinación de ambas capas es deliberada: la capa algebraica evita explorar el grafo cuando el problema no tiene solución, y la capa constructiva aporta la respuesta operativa que el usuario necesita.

5 Algoritmos, estructuras y modelos utilizados

5.1 Pseudocódigo — Euclides y Euclides extendido

```
función EUCLIDES(a, b): mientras b ≠ 0:
    (a, b) ← EUCLIDES(b, a mod b)
    devolver a
# a = gcd

función EUCLIDES_EXTENDIDO(a, b): (r0,
    r1) ← (a, b)
    (s0, s1) ← (1, 0)
    (t0, t1) ← (0, 1)
    # coef. de a #
    # coef. de b
    mientras r1 ≠ 0: q ←
        r0 div r1
        (r0, r1) ← (r1, r0 - q · r1) (s0, s1)
        ← (s1, s0 - q · s1) (t0, t1) ← (t1,
            t0 - q · t1)
    devolver (r0, s0, t0)
# gcd = a · s0 + b · t0
```

5.2 Pseudocódigo — Búsqueda en anchura (BFS)

```
función RESOLVER(n, m, k): inicio
  ← (0, 0) visitados ← { inicio }
  cola ← [ (inicio, [inicio]) ] mientras
  cola no esté vacía:
    ((x, y), camino) ← desencolar(cola) si x = k o
    y = k: devolver camino
    para cada operación op: # 6 sucesores nuevo ←
      aplicar(op, (x, y))
      si nuevo ∉ visitados:
        visitados ← visitados ∪ { nuevo } encolar(cola,
        (nuevo, camino + [nuevo]))
  devolver «sin solución»
```

5.3 Estructuras de datos y complejidad

- **Tuplas (x, y):** representan el estado; son inmutables y **hashables**, ideales como claves.
- **Conjunto de visitados:** evita reprocesar estados; consultas e inserciones en tiempo promedio $O(1)$.
- **Cola (deque):** implementa la frontera de la BFS con encolado y desencolado en $O(1)$.
- **Complejidad:** el espacio de estados tiene a lo sumo $(n + 1) \cdot (m + 1)$ nodos, por lo que la BFS es $O(n \cdot m)$ en tiempo y memoria; la decisión algebraica es $O(\log(\min(n, m)))$.

5.4 Implementación en Python

A continuación se presenta el núcleo funcional de la solución, que integra el criterio de solubilidad y la búsqueda de la secuencia mínima de operaciones.

```
from math import gcd
from collections import deque

def euclides_ext(a, b): old_r,
  r = a, b old_s, s = 1, 0
  old_t, t = 0, 1 while r !=
  0:
    q = old_r // r
    old_r, r = r, old_r - q * r old_s, s
    = s, old_s - q * old_t, t = t, old_t
    - q * t
  return old_r, old_s, old_t # gcd, coef_a, coef_b

def es_soluble(n, m, k):
  return (k % gcd(n, m) == 0) and (0 <= k <= max(n, m))

def resolver(n, m, k): inicio = (0,
  0) visitados = {inicio}
  cola = deque([(inicio, [(inicio, 'Inicio')])]) while cola:
    (x, y), camino = cola.popleft() if x == k
    or y == k:
      return camino
    t_ab = min(x, m - y) # A -> B
    t_ba = min(y, n - x) # B -> A sucesores =
    [
      ((n, y), 'Llenar A'), ((x, m), 'Llenar B'),
      ((0, y), 'Vaciar A'), ((x, 0), 'Vaciar B'), ((x - t_ab, y + t_ab),
      'Verter A->B'),
      ((x + t_ba, y - t_ba), 'Verter B->A'),
    ]
    for nuevo, accion in sucesores: if nuevo
      not in visitados:
        visitados.add(nuevo)
        cola.append((nuevo, camino + [(nuevo, accion)]))
  return None
```

6 Resultados obtenidos

Se ejecutó la implementación sobre un conjunto de casos de prueba que cubren situaciones solubles y no solubles. La tabla resume el máximo común divisor, el veredicto de solubilidad y el número mínimo de operaciones encontrado por la BFS.

n	m	k	$\gcd(n,m)$	¿Soluble?	Nº mín. operaciones
5	3	4	1	Sí	6
4	6	5	2	No	—
8	5	2	1	Sí	4
12	8	4	4	Sí	2
6	10	8	2	Sí	6

Tabla 1. Resultados sobre los casos de prueba.

6.1 Caso clásico: jarras de 5 y 3 litros, objetivo 4 litros

Como $\gcd(5, 3) = 1$ y $4 \leq \max(5, 3) = 5$, el problema es soluble. La búsqueda en anchura devuelve la secuencia mínima de **6 operaciones**:

Paso	Operación	Estado (A, B)
0	Inicio	(0, 0)
1	Llenar A	(5, 0)
2	Verter A $\rightarrow$ B	(2, 3)
3	Vaciar B	(2, 0)
4	Verter A $\rightarrow$ B	(0, 2)
5	Llenar A	(5, 2)
6	Verter A $\rightarrow$ B	(4, 3) ✓

Tabla 2. Trazo de la solución mínima para $(5, 3) \rightarrow 4$.

En el paso final, la jarra A contiene exactamente 4 litros, cumpliendo el objetivo. El algoritmo de Euclides extendido confirma la coherencia teórica: $5 \cdot (-1) + 3 \cdot (2) = 1$, y multiplicando por 4 se obtiene $5 \cdot (-4) + 3 \cdot (8) = 4$, una solución entera de la ecuación diofántica asociada.

6.2 Caso no soluble: jarras de 4 y 6 litros, objetivo 5 litros

Aquí $\gcd(4, 6) = 2$, que no divide a 5. Por el criterio de solubilidad, no existe ninguna secuencia de operaciones capaz de medir 5 litros: cualquier volumen alcanzable será necesariamente un número par. La capa algebraica detecta esta imposibilidad de inmediato, sin explorar el grafo de estados.

7

Limitaciones y posibles mejoras

Limitaciones

- El modelo aborda únicamente **dos** jarras; la generalización a tres o más recipientes incrementa notablemente el espacio de estados.
- La BFS almacena todos los estados visitados, con un costo de memoria $O(n \cdot m)$ que puede ser elevado para capacidades muy grandes.
- Se asume una fuente de agua ilimitada y operaciones sin pérdidas ni error de medición; no se contemplan restricciones físicas reales (derrames, volúmenes no enteros).
- El objetivo se busca en una sola jarra; no se modela el caso en que la cantidad k se distribuya entre ambas.

Posibles mejoras

- Emplear **BFS bidireccional** o el algoritmo A^* con una heurística admisible para reducir el número de estados explorados.
- Generalizar el modelo a N jarras y a objetivos distribuidos, apoyándose en la extensión de la identidad de Bézout a varios enteros.
- Reconstruir la secuencia de pasos directamente a partir de los **coeficientes de Bézout**, evitando la búsqueda cuando solo interesa una solución (no necesariamente la mínima).
- Construir una **interfaz gráfica** o una visualización animada del trasvase, en Python o JavaScript, para fines didácticos.
- Implementar el Pequeño Teorema de Fermat en la resolución de ecuaciones diofánticas lineales cuando sea aplicable, con el fin de facilitar el cálculo de las soluciones y mejorar la eficiencia del algoritmo.

8

Conclusiones

- El **algoritmo de Euclides** y la **identidad de Bézout** resuelven por completo la versión de decisión del problema: medir k litros es posible si y solo si $\gcd(n, m)$ divide a k y $k \leq \max(n, m)$.
- El problema de las dos jarras es un ejemplo tangible de cómo una **ecuación diofántica lineal** modela una situación cotidiana, conectando la teoría de números con la práctica.
- La estrategia de **dos capas** —decisión algebraica y construcción por búsqueda en anchura— combina eficiencia (descartar casos imposibles en tiempo logarítmico) con utilidad (entregar la secuencia mínima de pasos).
- El proyecto evidencia la transferibilidad de estos conceptos: el mismo aparato matemático sustenta la aritmética modular, la criptografía de clave pública y numerosos problemas de reparto y optimización.
- El proyecto evidencia la transferibilidad de estos conceptos: el mismo aparato matemático sustenta la aritmética modular, la criptografía de clave pública y numerosos problemas de reparto y optimización.

9

Glosario de teoremas y resultados nombrados

A continuación se recopilan, a modo de glosario, los teoremas y resultados matemáticos citados a lo largo del documento.

Teorema de la división entera

Para todo par de enteros a, b (con $b \neq 0$), existen enteros únicos q y r tales que $a = b \cdot q + r$, con

$0 \leq r < |b|$. Es la base sobre la que opera el algoritmo de Euclides al reemplazar (a, b) por $(b, a \bmod b)$.

Teorema de Euclides (algoritmo de Euclides)

$\gcd(a, b) = \gcd(b, a \bmod b)$, y $\gcd(a, 0) = a$. Aplicando esta identidad de forma repetida se obtiene el máximo común divisor en $O(\log(\min(a, b)))$ divisiones.

Identidad de Bézout

Para cualesquiera enteros a y b , existen enteros x , y (coeficientes de Bézout) tales que $a \cdot x + b \cdot y = \gcd(a, b)$. Además, el conjunto de todos los valores expresables como $a \cdot x + b \cdot y$ es exactamente el conjunto de múltiplos de $\gcd(a, b)$.

Algoritmo de Euclides extendido

Variante del algoritmo de Euclides que, además de calcular $\gcd(a, b)$, obtiene explícitamente los coeficientes x , y de la identidad de Bézout, arrastrando las combinaciones lineales durante las divisiones sucesivas.

Teorema de existencia de soluciones de una ecuación diofántica lineal

La ecuación $a \cdot x + b \cdot y = c$ tiene solución entera si y sólo si $\gcd(a, b)$ divide a c . Si (x_0, y_0) es una solución particular y $d = \gcd(a, b)$, todas las soluciones se generan mediante $x = x_0 + (b/d) \cdot t$, $y = y_0 - (a/d) \cdot t$, con $t \in \mathbb{Z}$.

Criterio de solubilidad del problema de las dos jarras

Es posible medir exactamente k litros con jarras de capacidades n y m si y sólo si: (1) $\gcd(n, m)$ divide a k , y (2) $k \leq \max(n, m)$. Es una aplicación directa de la identidad de Bézout más una restricción física.

Teorema de Lamé

Relaciona el número de pasos del algoritmo de Euclides con la sucesión de Fibonacci: el peor caso (mayor número de divisiones para números de cierto tamaño) ocurre con pares de Fibonacci consecutivos.

Pequeño teorema de Fermat

El Pequeño Teorema de Fermat puede emplearse para obtener una solución particular de la ecuación diofántica lineal cuando uno de los coeficientes a o b es un número primo y se utiliza como módulo. Este teorema establece que:

$$a^{b-1} \equiv 1 \pmod{b} \rightarrow a^{b-2} \equiv a^{-1} \pmod{b}$$

Al reducir la ecuación diofántica lineal módulo b , se obtiene:

$$ax + by = c \rightarrow ax \equiv c \pmod{b} \rightarrow x \equiv c a^{b-2} \pmod{b}$$

A partir de esta identidad es posible obtener el inverso multiplicativo de a , lo que permite calcular la solución de la congruencia asociada al problema mediante:

$$x = c a^{b-2} \pmod{b}$$

De esta manera, se simplifica el cálculo de los valores de x y y de la ecuación diofántica lineal cuando se cumplen las condiciones de aplicación del teorema.

Education.

- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). **Introduction to Algorithms** (3.^a ed.). The MIT Press.
- [3] Niven, I., Zuckerman, H. S. y Montgomery, H. L. (1991). **An Introduction to the Theory of Numbers** (5.^a ed.). John Wiley & Sons.
- [4] Knuth, D. E. (1997). **The Art of Computer Programming, Vol. 2: Seminumerical Algorithms** (3.^a ed.). Addison-Wesley.
- [5] Grimaldi, R. P. (2003). **Discrete and Combinatorial Mathematics: An Applied Introduction** (5.^a ed.). Pearson Addison-Wesley.
- [6] Graham, R. L., Knuth, D. E. y Patashnik, O. (1994). **Concrete Mathematics: A Foundation for Computer Science** (2.^a ed.). Addison-Wesley.
- [7] Python Software Foundation. (2024). **math — Mathematical functions** [Documentación oficial de Python 3]. <https://docs.python.org/3/library/math.html>
- [8] Python Software Foundation. (2024). **collections — Container datatypes (deque)** [Documentación oficial de Python 3]. <https://docs.python.org/3/library/collections.html>
- [9] [9] De Nápoli, P. L. (2020). El Teorema Chino del Resto y El Pequeño Teorema de Fermat. [Material de clase, Álgebra I]. Departamento de Matemática, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires.